Edge Preserving Image Denoising : kNN vs. Averaging

To compare the efficiency and quality of k-Nearest Neighbor (kNN) intensity filtering against traditional mean filtering using MSE (Mean Squared Error) as a benchmark.

Presenter: Vaibhav Rathore (25D1387)

GNR 630 : Introduction to Geospatial Technologies

Course Instructor : Prof. B.K Mohan

Presentation Overview

01
Problem Statement

02
Dataset Overview

03
Methodology

04
Implementation Details

05
Results and Analysis

06
UI Demonstration

07
Conclusion

Problem Statement

The project addresses the critical trade-off in image processing between **noise reduction** and **detail preservation**.

The Problem

Standard smoothing filters (like Mean filters) reduce noise but often blur critical edges, making images look "out of focus."

THE TASK

01

Add uniform noise to a clean benchmark image.

02

kNN Algorithm: Average only the k most similar pixels in a local window.

03

Compare result against **Simple Averaging** using identical window sizes.

04

Quantify performance using **Mean Squared Error (MSE)**.

Dataset Overview

To ensure the algorithm is robust across different visual contexts, we utilize a curated set of standard benchmark images from the `scikit-image` library.

Camera (Grayscale)

Essential for testing edge preservation without the complexity of color.

Astronaut (Color)

High-detail RGB image for testing multi-channel processing.

Chelsea (Color)

Features fine textures (cat fur) to test blurring vs. denoising.

Coffee (Color)

Focuses on gradients and smooth surfaces.

PURPOSE: These standard images allow for repeatable, objective comparisons between the kNN and Averaging filters.

File Upload Support

- Users can upload their own images (PNG, JPG, BMP) via the Streamlit interface.
- **Function:** `load_image_from_bytes` handles raw data streams directly from the browser.

Algorithmic Description: Simple Averaging

The **Averaging Filter** (or Mean Filter) is the most basic form of smoothing.

Mechanism

For every pixel, it looks at a (W×W) neighborhood and calculates the mathematical mean of all pixels.

Behavior

It acts as a low-pass filter.

Trade-off

While it reduces noise effectively, it significantly **blurs edges** because it treats "edge pixels" and "noise pixels" identically.

Algorithmic Description: kNN Smoothing

The **k-Nearest Neighbor (kNN)** algorithm is a more "intelligent" approach to smoothing.

Mechanism

1. Define a window (e.g., 5×5).
2. Identify the **center pixel's** intensity.
3. Calculate the distance between the center and all neighbors.
4. Select k pixels **closest in intensity**.
5. Average only those k values.

Advantage

It **preserves edges** better because it avoids averaging the center pixel with neighbors that look significantly different (i.e., pixels on the other side of an edge).

Measuring Success: Mean Squared Error

To quantify "how much better" one filter is over the other, we use **Mean Squared Error (MSE)**. It calculates the average of the squares of the errors between the original image (I) and the denoised image (K).

The Formula

$$MSE = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I(i,j) - K(i,j)]^2$$

Lower MSE

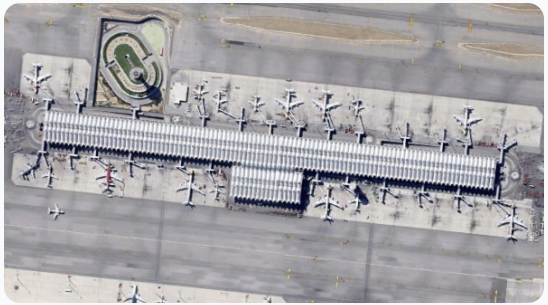
Indicates a denoised image that is numerically closer to the original.

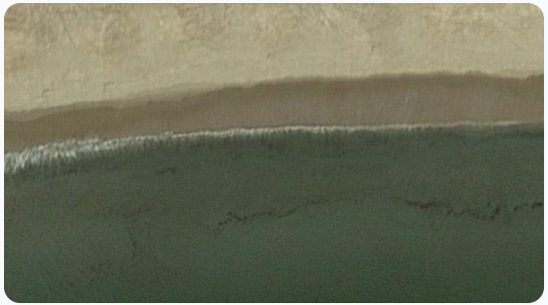
Higher MSE

Suggests either remaining noise or significant blurring and distortion.

Sample Data and Results

A representative collection of test datasets used to evaluate the **edge-preserving** performance of the algorithm across diverse scenes.





UI Implementation

Upload an image

baseballfield_2.jpg
117.6KB

+

Noise amplitude
0.08

k (kNN neighbors)
5

Window size
5
3 21

Max image dimension for speed
256

Random seed
42 – +

Show MSE vs k curve

Curve max k
15

Run denoising

Image Denoising with kNN vs Averaging Filter

Uniform noise injection, local-window kNN denoising, averaging filter baseline, and MSE comparison.

Input and outputs

Original

Noisy (A=0.08)

kNN denoised MSE=0.001913

Average denoised MSE=0.001309

Noisy MSE	kNN MSE	Average MSE	Image size
0.002124	0.001913	0.001309	256 x 170

Runtime

kNN: 0.764 s

Average: 0.002 s

Conclusion

kNN is "Edge-Aware"

By selecting pixels based on intensity similarity, kNN provides a much sharper denoised image than simple averaging.

The Cost of Quality

The primary drawback of kNN is its computational complexity. While averaging can be done via fast Fourier transforms, kNN requires local sorting/partitioning.

Final Verdict

kNN is the superior choice for applications where image clarity and edge retention are more critical than processing speed.

Thank you